Web Development Foundations

Week 5 - DOM: "Now It Connects"

Day 2: "DOM Input Feedback"

1

Reconnect

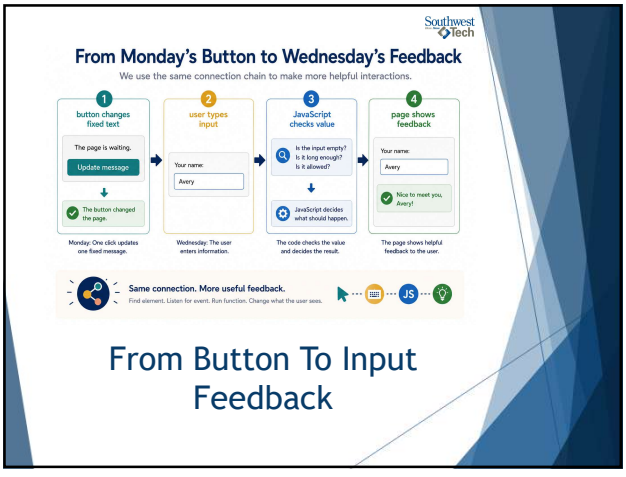
Monday:

- aselected page elements
- waited for a click
- ran a function
- changed visible text

Today:

- read user input
- check the value
- give clearer feedback

2



From Button To Input Feedback

3

The Working Problem: Fixed Text Is Limited

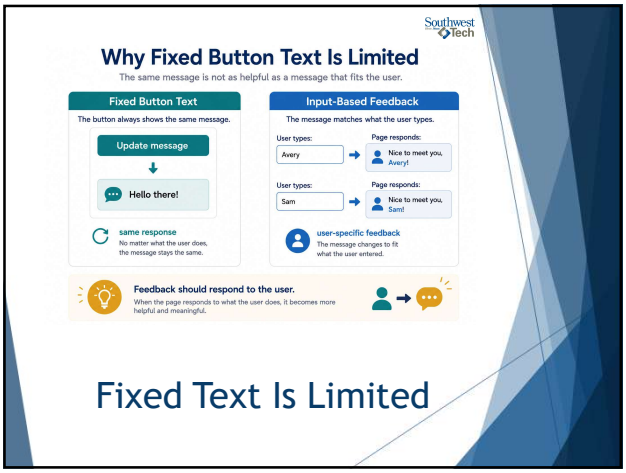
A fixed message proves the connection works.

But users often need feedback based on what they do.

Examples:

- empty input needs a reminder
- typed input can personalize a message
- unclear feedback leaves users guessing

4



Fixed Text Is Limited

5

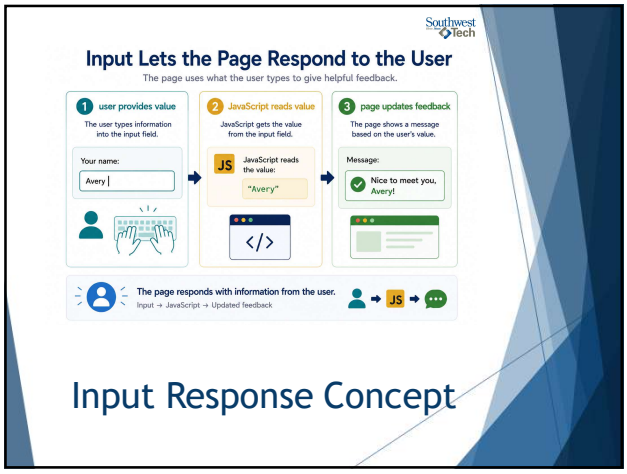
What Counts as Success Today

- an input is selected
- the current `.value` is read
- empty input is handled
- feedback appears on the page
- selector mistakes are easier to spot

6


Inputs Make The Page Respond To The User
Input changes the interaction.
Without input:
the page gives the same response every time
With input:
the user provides a value
JavaScript reads that value
the page responds with feedback

7


Southwest Tech
Input Lets the Page Respond to the User
The page uses what the user types to give helpful feedback.
1 user provides value
The user types information into the input field.
Your name:
Avery
2 JavaScript reads value
JavaScript gets the value from the input field.
JS
JavaScript reads the value:
"Avery"
3 page updates feedback
The page shows a message based on the user's value.
Message:
Nice to meet you, Avery!
The page responds with information from the user.
Input → JavaScript → Updated feedback
Input Response Concept

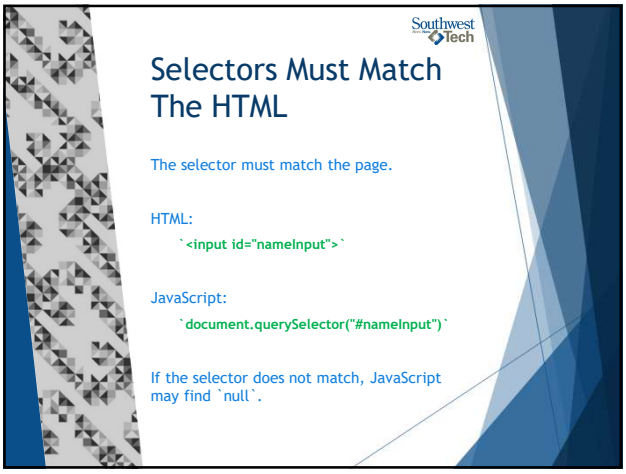
8


Southwest Tech
What We Will Use Today
input element
label
`.value`
`.trim()`
empty string
`return`
feedback message
selector check

9


Southwest Tech
Today's Input–Feedback Toolbox
The tools we need to read input and show helpful feedback.
input
label
.value
.trim()
empty string
return
feedback message
selector check
TODAY'S TOOLS
Small tools. Big impact.
User
JS
Feedback
Input Feedback Toolbox

10


Southwest Tech
Selectors Must Match The HTML
The selector must match the page.
HTML:
`<input id="nameInput">`
JavaScript:
`document.querySelector("#nameInput")`
If the selector does not match, JavaScript may find `null`.

11


Southwest Tech
A JavaScript Selector Must Match the HTML
JavaScript finds elements by their id.
HTML
The input element in the page.
JavaScript
JavaScript uses a selector to find the element.
Your name:
Type your name...
id = nameInput
This selector looks for an element with id = nameInput
No match can mean null.
If the id in your HTML and the selector in your JavaScript do not match exactly, querySelector() returns null.
IDs are case-sensitive.
Check spelling carefully.
Check the # in your selector.
Selector Must Match HTML

12

Reading Input Values

Selecting the input finds the element.

- `.value` reads what the user typed.

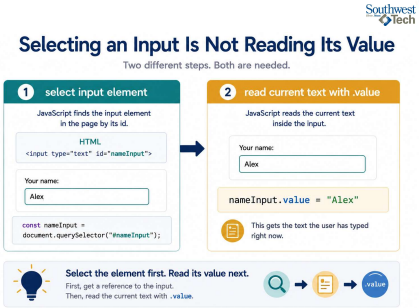
Example:

```
const name = nameInput.value.trim();
```

Read it as:

"Get the current input text and remove extra spaces."

13



Reading Input Values

14

Feedback Decision Branch

Good feedback often needs a condition.

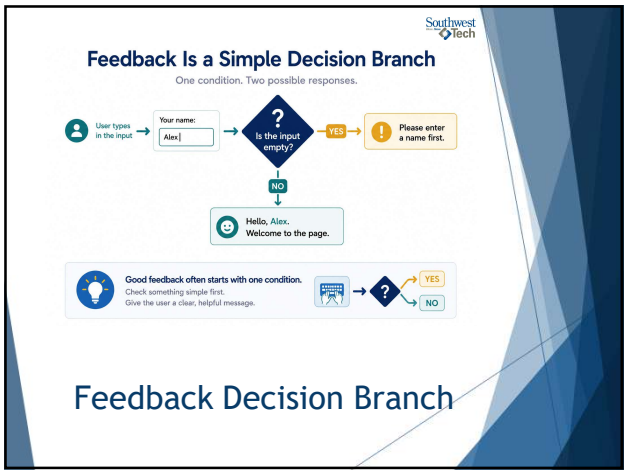
If the input is empty:

- ask the user to enter something
- stop the function

Otherwise:

- use the input
- update the message

15


Southwest Tech
Feedback Is a Simple Decision Branch
One condition. Two possible responses.
User types in the input
Your name:
Alex
Is the input empty?
YES
Please enter a name first.
NO
Hello, Alex.
Welcome to the page.
Good feedback often starts with one condition.
Check something simple first.
Give the user a clear, helpful message.
YES
NO
Feedback Decision Branch

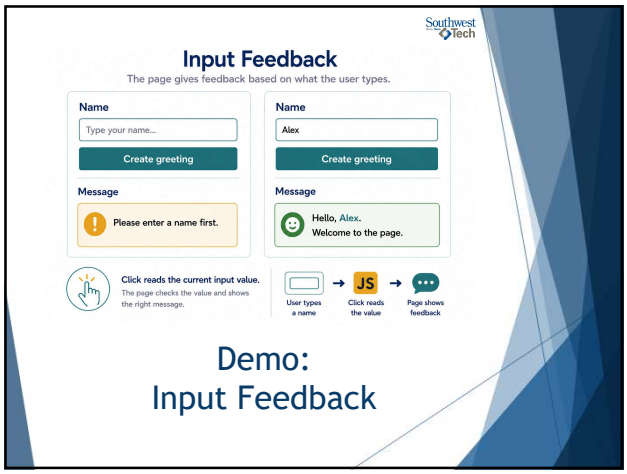
16


Demo
"Input Feedback"

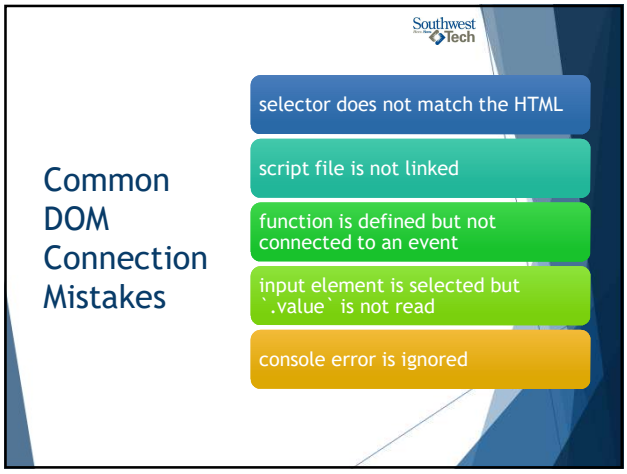
17


Southwest Tech
What to Watch For
input, button, and message selected
`.value` read after the click
empty input checked
feedback message updated
click event still controls timing
Q2
Q3

18


Southwest Tech
Input Feedback
The page gives feedback based on what the user types.
Name
Type your name...
Create greeting
Message
Please enter a name first.
Name
Alex
Create greeting
Message
Hello, Alex.
Welcome to the page.
Click reads the current input value.
The page checks the value and shows the right message.
User types a name
JS
Click reads the value
Page shows feedback
Demo:
Input Feedback

19


Southwest Tech
Common DOM Connection Mistakes
selector does not match the HTML
script file is not linked
function is defined but not connected to an event
input element is selected but `.value` is not read
console error is ignored

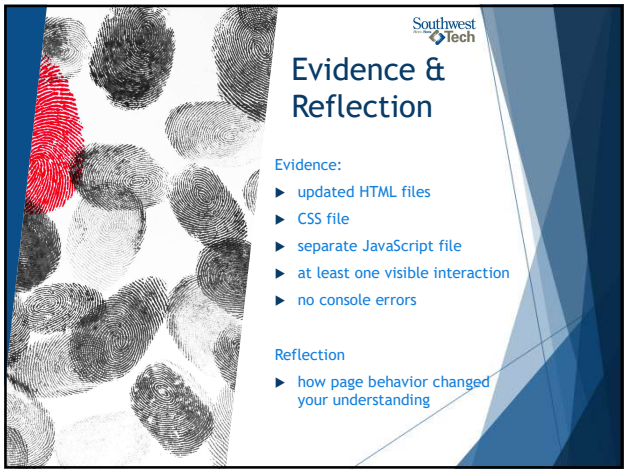
20


Southwest Tech
Common Beginner DOM Connection Mistakes
Use this checklist to find and fix the most common issues.
1 selector does not match HTML
Check id names, # vs ., and spelling. Even one character can break the connection.
2 script file is not linked
Make sure the <script src="..."></script> is in your HTML file.
3 function is not connected to an event
Add an event (like click) that calls your function.
4 input selected but .value not read
Select the element first. Then read the current text with .value.
5 console error ignored
Console messages show what went wrong and where to look.
Connection problems leave clues.
Read carefully, check patiently, and test again.
Small checks lead to working pages!
Common DOM Connection Mistakes

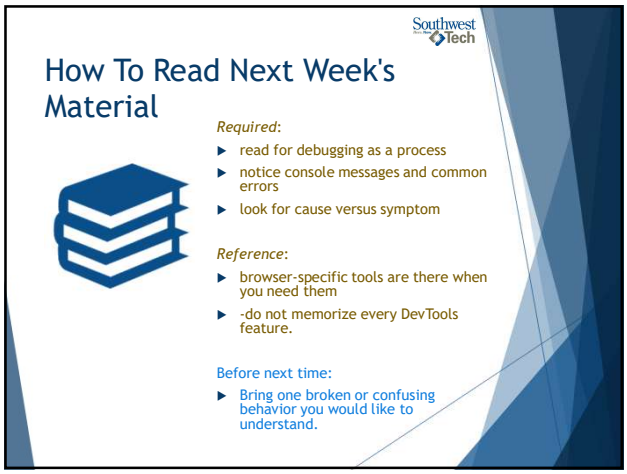
21


Southwest Tech
Lab Refinement
Your goal:
keep the JavaScript in a separate file
make one interaction reliable
add a second interaction or improve the first
improve user feedback
remove console errors

22


Southwest Tech
Evidence & Reflection
Evidence:
updated HTML files
CSS file
separate JavaScript file
at least one visible interaction
no console errors
Reflection
how page behavior changed your understanding

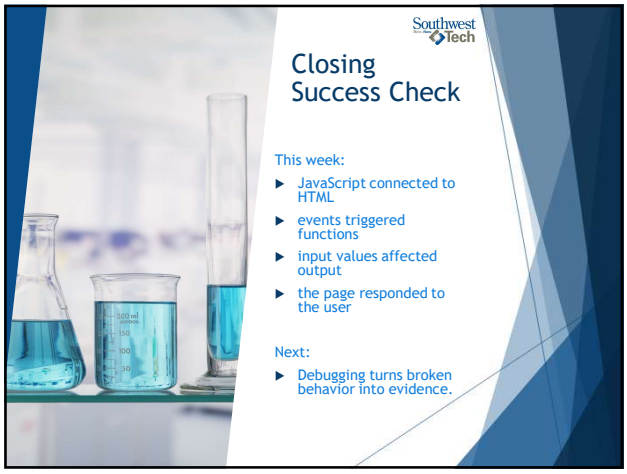
23


Southwest Tech
How To Read Next Week's Material
Required:
read for debugging as a process
notice console messages and common errors
look for cause versus symptom
Reference:
browser-specific tools are there when you need them
-do not memorize every DevTools feature.
Before next time:
Bring one broken or confusing behavior you would like to understand.

24


From Reactive Pages to Debugging
A natural next step in your development journey.
1 page responds
The page works as expected.
2 behavior breaks
Something changes and it stops working.
3 browser gives clues
The browser provides helpful information.
4 debugging process begins
Use the clues, test, and fix the issue.
When behavior breaks, evidence matters.
Observe → Collect clues → Understand → Fix → Verify
Reactive Page
To Debugging Bridge

25


Closing
Success Check
This week:
JavaScript connected to HTML
events triggered functions
input values affected output
the page responded to the user
Next:
Debugging turns broken behavior into evidence.

26


Thank you!
Have Fun!

27